

Πανεπιστήμιο Κρήτης, Τμήμα Επιστήμης Υπολογιστών
HY252 – Αντικειμενοστρεφής Προγραμματισμός
Εξάμηνο: **Χειμερινό 2025-2025**
Διδάσκων: **Γιάννης Τζίτζικας**
Βοηθοί: 1η (B10), 2η (B11), 3η (B15), 4η (B7), 5η (B8), 6η (B5)

2η Σειρά Ασκήσεων

Ανάθεση: 21 Οκτωβρίου 2025

Παράδοση: 14 Νοεμβρίου 2025

(δείτε τις ημερομηνίες για **bonus** που περιγράφονται παρακάτω)

Εκπαιδευτικοί στόχοι

Αυτή η σειρά ασκήσεων είναι ίσως η πιο σημαντική του μαθήματος.

- *Η 1η άσκηση (**αισθητήρες,** 20 μονάδες) θα σας επιτρέψει να εξοικειωθείτε με τον ορισμό κλάσεων και υποκλάσεων, τη δημιουργία αντικειμένων (στιγμιότυπων των κλάσεων που ορίσατε) και την υποσκέλιση μεθόδων (overriding)*
- *Η 2η άσκηση (**σύστημα συναγερμού,** 20 μονάδες) θα σας επιτρέψει να εξοικειωθείτε με την προδιαγραφή Αφαιρετικών Τύπων Δεδομένων (Abstract Data Types), με φυσική γλώσσα, και με pre/post conditions και invariants, και την υλοποίησή τους με κλάσεις Java (και χρήση εξαιρέσεων), την τεκμηρίωση τους με JavaDoc και τον έλεγχο/δοκιμή τους με Junit tests*
- *Η 3η άσκηση (**Γραφική Κονσόλα Συναγερμού**, 20 μονάδες) θα σας επιτρέψει να έρθετε σε μια πρώτη επαφή με τα γραφικά της Java (που θα σας χρειαστούν και στο project)*
- *Η 4η άσκηση (**Νήματα**, 5 μονάδες) θα σας επιτρέψει να εξασκηθείτε με τα νήματα (threads)*
- *Η 5η άσκηση (**Star Wars**, 35 μονάδες) θα σας επιτρέψει να εξοικειωθείτε με τις διεπαφές (interfaces), τις αφηρημένες (abstract) κλάσεις και τα συνεργαζόμενα αντικείμενα (Collaborating Objects)*
- *Η 6η άσκηση (**Android**, 20 μονάδες) είναι μια προαιρετική άσκηση που δίνει extra μονάδες*

Σημειώσεις

1. **Απορίες** σχετικά με την Α2 θα απαντώνται μόνο μέσω του forum της Α2 στο moodle. Επίσης μην ξεχνάτε ότι κάθε Παρασκευή υπάρχουν ώρες γραφείου.
2. Θα γίνει και **φροντιστήριο** σχετικό με JavaDoc και JUnit. Τα generics θα διδαχθούν αναλυτικά αργότερα, στην παρούσα φάση απλά θα τα χρησιμοποιήσετε.
3. **Ημερομηνία παράδοσης** και **bonus**: Όσοι παραδώσουν τις **ασκήσεις 1, 2 στις 12 Νοεμβρίου** θα έχουν **10% bonus** στο βαθμό που πήραν σε αυτές τις δύο ασκήσεις. Όσοι παραδώσουν τις **ασκήσεις 3, 4, 5 στις 23 Νοεμβρίου** θα έχουν **bonus 10%** σε αυτές τις ασκήσεις. Διευκρίνηση: Αν κάποιος παραδώσει τις ασκήσεις 1, 2 και στο submit με το bonus και στο submit χωρίς bonus τότε **μετράει το τελευταίο (το χωρίς bonus).** Το ίδιο ισχύει και για τις ασκήσεις 3, 4, 5.

Άσκηση	*Χρονικό Διάστημα*
1η και 2η	22 Οκτ. – 5 Νοεμ. (bonus)
3η, 4η ,5η	4 – 14 Νοεμ. (12 Νοεμβρίου για bonus)
6η	Αυτήν μπορείτε να την παραδώσετε το Δεκέμβριο (θα βγει ανακοίνωση)

4. **Οδηγίες Παράδοσης**. **ΠΡΟΣΟΧΗ: Ακολουθήστε επακριβώς τις οδηγίες παράδοσης που αναγράφονται στο έγγραφο "Οδηγίες Παράδοσης Ασκήσεων" στο elearn. Διαφορετικά η διόρθωση είναι πολύ χρονοβόρα για τους μεταπτυχιακούς οι οποίοι έχουν και αυτοί μαθήματα και εργασία να κάνουν. Αν παραδώστε κάτι που δεν είναι σύμφωνο με τους κανόνες παράδοσης τότε θα υπάρξει μείωση βαθμού και ίσως δεν βαθμολογηθεί.**
5. Αφού κατεβάσετε τον φάκελο project templates για την 2η σειρά ασκήσεων από το elearn, επιλέξτε το template που αντιστοιχεί στο IDE που θα χρησιμοποιήσετε και **αναπτύξτε τις ασκήσεις της 2ης σειράς (εκτός από την προαιρετική άσκηση 6) πάνω σε αυτό το template**.
6. Η **εξέταση** της Α2 θα γίνει μέσω μίας υποχρεωτικής εργαστηριακής άσκησης **(A2E**). Θα είναι μια απλή και μικρή άσκηση που θα πρέπει να κάνετε στο εργαστήριο/αναγνωστήριο (θα δούμε εφέτος πως θα την κάνουμε – ανάλογα με τις συνθήκες) υπό την εποπτεία των βοηθών του μαθήματος (σε περίπτωση απουσίας ή αποτυχίας δεν θα προσμετρηθούν οι βαθμοί της Α2 – για ειδικές περιπτώσεις (π.χ. απουσίες λόγω θεμάτων υγείας) υπάρχει σχετικό forum).

CC BY NC SA



Άσκηση 1. Αισθητήρες Αξία: **20 μονάδες**
Δημιουργία αντικειμένων, overriding, συνδεδεμένες κλάσεις

Θέλουμε να φτιάξουμε κλάσεις για αισθητήρες συστημάτων συναγερμού, και για γραμμές στις οποίες έχουμε συνδέσει πολλούς τέτοιους αισθητήρες.

(α) [5 μονάδες]

(α.1) Δημιουργείστε μια κλάση **Sensor** στο πακέτο A21. Πρέπει να έχει τρία ιδιωτικά πεδία **id** (String), **violation** (boolean), **on** (Boolean). Συμπληρώστε τις μεθόδους που απαιτούνται βάσει της αρχής της *ενθυλάκωσης*. Ορίστε κατάλληλα μια κατασκευάστρια μέθοδο που να λαμβάνει τις κατάλληλες παραμέτρους και εσωτερικά να καλεί τις αντίστοιχες set-μεθόδους. Ορίσετε μία δημόσια μέθοδο String **toString()** η οποία θα επιστρέφει ένα string με τις τιμές των πεδίων με κατάλληλη μορφή για ανάγνωση. Για παράδειγμα ο κώδικας

```
class   SensorClient {
      public static void main(String[] lala) {
            System.out.println(new Sensor("S01",false,false));
      }
}
```

πρέπει να επιστρέφει

```
Sensor with id=S01, On state = false, Violation state = false
```

(α.2)

Δημιουργήστε (στο ίδιο πακέτο) μια υποκλάση **TemperatureSensor** της Sensor χωρίς κανένα επιπλέον πεδίο, όμως πρέπει να έχει μια κατασκευάστρια μέθοδο που να καλεί την κατασκευάστρια της υπερκλάσης και να κάνει override τη μέθοδο toString ώστε να επιστέφει ό,τι και η toString της γονικής συν τον τύπο του αισθητήρα. Επίσης δημιουργήστε (στο ίδιο πακέτο) άλλη μια υποκλάση **LaserSensor** της Sensor η οποία πρέπει να έχει ένα επιπλέον ιδιωτικό πεδίο range (float), εκ τούτου προσθέστε τις set/get μεθόδους που απαιτούνται. Να έχει μια κατασκευάστρια μέθοδο η οποία να λαμβάνει 4 παραμέτρους (τις 3 της γονικής συν την τιμή για την ιδιότητα range) η οποία να καλεί την κατασκευάστρια της υπερκλάσης για τις 3 πρώτες παραμέτρους και να θέτει την τιμή της εμβέλειας. Κάντε override τη μέθοδο toString ώστε να επιστέφει ό,τι και η toString της γονικής συν την εμβέλεια και τον τύπο του αισθητήρα, ώστε για παράδειγμα ο κώδικας

```
class   SensorTESTERsimple {
      public static void main(String[] lala) {
            System.out.println(new Sensor("S01",false,false));
            System.out.println(new TemperatureSensor("S02",false,false));
            System.out.println(new LaserSensor("S03",false,false,4F));
      }
}
```

να επιστρέφει

```
Sensor with id=S01, On state = false, Violation state = false
Sensor with id=S02, On state = false, Violation state = false, (temp. sensor)
Sensor with id=S03, On state = false, Violation state = false, range = 4.0, (laser sensor)
```

(β) [10 μονάδες]

Θέλουμε τώρα να φτιάξουμε μια *γραμμή* η οποία να έχει διάφορους αισθητήρες. Για το σκοπό αυτό δημιουργείστε μια κλάση **SensorLine**. Κάθε στιγμιότυπο της πρέπει να έχει ένα όνομα που θα περνάει από την κατασκευάστρια μέθοδο. Επίσης να έχει μια κατασκευάστρια μέθοδο χωρίς όρισμα (σε αυτήν την περίπτωση το default όνομα να είναι “Anonymous sensorLine". Επίσης πρέπει να έχει ως ιδιωτικό πεδίο έναν πίνακα (1000 θέσεων) που θα μπορεί να δείχνει σε αντικείμενα τύπου Sensor. Πρέπει η κλάση αυτή να προσφέρει μια **add(Sensor s)** η οποία να προσθέτει έναν αισθητήρα **s** στον πίνακα (αρκεί να υπάρχει χώρος στον πίνακα). Για το λόγο αυτό πρέπει να έχει ένα ιδιωτικό πεδίο **activeCapacity** (int) (αρχικά με τιμή 0) που να αυξάνεται κατά 1 μετά από κάθε add. Επίσης η κλάση πρέπει να προσφέρει μια μέθοδο **setOn(Boolean)** η οποία αντίστοιχα θα ανοίγει/κλείνει *όλους* τους αισθητήρες που έχει στον πίνακά της. Επίσης πρέπει να προσφέρει μια **boolean isViolated ()** που να επιστρέφει true όταν τουλάχιστον ένας αισθητήρα είναι violated. Επίσης η κλάση πρέπει να κάνει override την toString() η οποία να επιστρέφει μια συμβολοσειρά με τα εξής στοιχεία:

(α) το πλήθος των sensors που έχει ο πίνακάς της

(β) αν είναι violated ή όχι

(γ) το πλήθος αυτών που έχουν violated:on

(δ) τα ids αυτών που έχουν violation on.

Τέλος πρέπει να προσφέρει μια μέθοδο **getSensors()** που να επιστρέφει έναν πίνακα με τους πραγματικούς αισθητήρες που έχει.

Εμπλουτίστε την SensorTester ωστε να δημιουργεί μια SensorLine με δύο αισθητήρες (έναν απλό και έναν laser) και δοκιμάστε την. Η εκτύπωση της toString μπορεί να επιστέφει κάτι της μορφής:

```
SensorLine Anonymous sensorLine
Number of sensors=2
Violated=true
# of violated sensors=1
Ids of Violated: S02
```

γ) [5 μονάδες] // Client

Εμπλουτίστε τη μέθοδο main της κλάσης SensorTester ώστε να φτιάνει ένα στιγμιότυπο της SensorLine στο οποίο να προσθέτει 100 αισθητήρες, ο 1ος αισθητήρας να είναι απλός, ο 2ος θερμοκρασίας, ο 3ος laser, κ.ο.κ, Τα αναγνωριστικά τους να είναι SID1, …, SID100. Οι ακτίνες των αισθητήρων laser να είναι όσο ο αντίστοιχος δείκτης του πίνακα. Επίσης το 1/4 από αυτούς να είναι violated. Στο τέλος να καλεί toString αυτής της SensorLine η οποία θα επιστρέφει κάτι της μορφής:

```
SensorLine Anonymous sensorLine
Number of sensors=100
Violated=true
# of violated sensors=25
Ids of Violated: SID0 SID4 SID8 SID12 SID16 SID20 SID24 SID28 SID32 SID36 SID40
SID44 SID48 SID52 SID56 SID60 SID64 SID68 SID72 SID76 SID80 SID84 SID88 SID92 SI
D96
```



Άσκηση 2. Ελεγκτής Συστήματος Συναγερμού

Αξία: **20 μονάδες**

Προδιαγραφή και υλοποίηση Κλάσεων, Εξαιρέσεις

Θέλουμε να φτιάξουμε το λογισμικό ενός **ελεγκτή** (controller) ενός κλασσικού συστήματος οικιακού συναγερμού, ο οποίος θα συνδεθεί με την κονσόλα (πληκτρολόγιο) του συναγερμού και με 2 ή περισσότερες γραμμές αισθητήρων τύπου **SensorLine** (από την προηγούμενη άσκηση). Για το σκοπό αυτό καλείστε να σχεδιάσετε και να υλοποιήσετε έναν Αφαιρετικό Τύπο Δεδομένων ADT (Abstract Data Type) με το όνομα **HomeSecurityADT** στο πακέτο A22.

Ο ADT πρέπει να επιτρέπει να δημιουργήσουμε αντικείμενα τέτοιου τύπου. Η κατασκευάστρια μέθοδος πρέπει να λαμβάνει ως παράμετρο **δύο** πίνακες από στιγμιότυπα τύπου **SensorLine** (από την προηγούμενη άσκηση). Ο πρώτος πίνακας θα έχει γραμμές αισθητήρων που αφορούν το εσωτερικό του σπιτιού (π.χ. ραντάρ κίνησης), ενώ ο δεύτερος πίνακας θα έχει γραμμές που σχετίζονται με το εξωτερικό του σπιτιού (π.χ. αισθητήρες επαφής για πόρτες και παράθυρα, εξωτερικά ραντάρ, κ.α.).

Επίσης πρέπει να προσφέρει μεθόδους που να επιτρέπουν:

- Να δούμε την κατάστασή του συναγερμού (αν είναι οπλισμένος, αφοπλισμένος, ή σε Stay mode – Το Stay mode είναι μια κατάσταση λειτουργίας όπου είναι ενεργοποιημένοι μόνο οι αισθητήρες για το εξωτερικό του σπιτιού, ενώ οι εσωτερικοί είναι απενεργοποιημένοι, άρα οι κάτοικοι του σπιτιού μπορούν να κινούνται μέσα στο σπίτι χωρίς να ενεργοποιείται ο συναγερμός)
- Να οπλίσουμε (Arm) το συναγερμό, αρκεί όλες οι γραμμές αισθητήρων (εσωτερικές και εξωτερικές) να μην είναι violated
- Να ενεργοποιήσουμε την επιλογή Stay, αρκεί όλες οι γραμμές εξωτερικών αισθητήρων να μην είναι violated, και ο συναγερμός είναι αφοπλισμένος
- Να απενεργοποιήσουμε είτε το Arm ή το Stay, αρκεί να δώσουμε το σωστό κωδικό
- Να αλλάξουμε τον κωδικό, αρκεί ο συναγερμός να είναι αφοπλισμένος

Η δημιουργία ενός αντικειμένου πρέπει να έχει default κωδικό το “1111”.

α) [5 μονάδες]

Δώστε τις υπογραφές των μεθόδων του ADT HomeSecurity (διακρίνοντας τις σε **constructors**, **accessors**, **transformers**, **observers**) και τις προ/μετα-συνθήκες (pre/post-conditions) κάθε μεθόδου (στα αγγλικά) ορίζοντας μια διεπαφή (interface) ή αφηρημένη κλάση (abstract class). Επίσης προσπαθήστε να δώσετε κάποιες αναλλοίωτες συνθήκες (invariants) εκφράζοντας τις ως εξισωτικά αξιώματα (equational axioms) στα σχόλια. Τα παραπάνω περιλαμβάνουν τον προσδιορισμό των εξαιρέσεων (exceptions) και που και πότε θα εγείρονται. Θα πρέπει να υποστηρίζονται τουλάχιστον, οι παρακάτω εξαιρέσεις:

InternalSensorViolationException
ExternalSensorViolationException
WrongPasswordException
NotDisarmedException

Τεκμηριώστε πλήρως την προδιαγραφή σας με χρήση **Javadoc** (δείτε τις οδηγίες στο σχετικό φροντιστήριο). Πλοηγηθείτε με ένα πρόγραμμα περιήγησης ιστού για να βεβαιωθείτε ότι η παραχθείσα τεκμηρίωση είναι εντάξει.

β) [8 μονάδες]

Δώστε μια υλοποίηση του ADT σε Java και μην λησμονήσετε να ορίσετε τις απαιτούμενες εξαιρέσεις.

γ) [5 μονάδες]

Προσθέστε κώδικα που να ελέγχει ότι η υλοποίηση του σκέλους β) είναι σωστή βάσει της προδιαγραφής που δώσατε στο σκέλος α). Αρχικά, για διευκόλυνση σας, βάλτε τον κώδικα σε μια κλάση με όνομα HomeSecurityMyTest. Ο κώδικας αυτός πρέπει να περιλαμβάνει σενάρια της μορφής:

1. δημιουργία συναγερμού με δύο γραμμές αισθητήρων
2. οπλισμός
3. αφοπλισμός με τον σωστό κωδικό
4. αλλαγή κωδικού
5. ενεργοποίηση stay
6. disarm με το νέο κωδικό

Κατόπιν αυτά τα σενάρια ελέγχου (γράψτε τουλάχιστον 3 σενάρια) να τα κάνετε **JUnit tests** (διαβάστε το σχετικό κεφάλαιο του βιβλίου «Μια Γεύση από Java" και δείτε και το υλικό του φροντιστηρίου), ώστε να έχουμε αυτοματοποιημένους ελέγους ορθότητας λογισμικού. Τοποθετήστε την κλάση με αυτούς τους ελέγχους στο ίδιο πακέτο με όνομα **HomeSecurityTest.java** και δοκιμάστε την (Run as Junit tests).

δ) [2 μονάδες]
Καλό είναι τα Arm, Disarm και Stay να συνοδεύονται από κάποιο χαρακτηριστικό ήχο. Πέραν του κλασσικού beep που μπορείτε να κάνετε με την κλήση

```
java.awt.Toolkit.getDefaultToolkit().beep();
```

καλείστε να χρησιμοποιείστε την κλάση Player που ορίζεται σε ένα jar με όνομα jfugue-4.0.3.jar που σας δίνεται στο moodle. Για το λόγο αυτό πρέπει να το προσθέσετε στη βιβλιοθήκη του project σας αυτό το jar, και στον κώδικα σας το εξής import::

- import org.jfugue.Player;

Για να παίξετε τη νότα που αντιστοιχεί σε έναν ακέραιο (από το 0 έως το 127), ας πούμε για τον ακέραιο 80, αρκούν τα παρακάτω:

```
Player p = new Player();
p.play("[80] ");
```

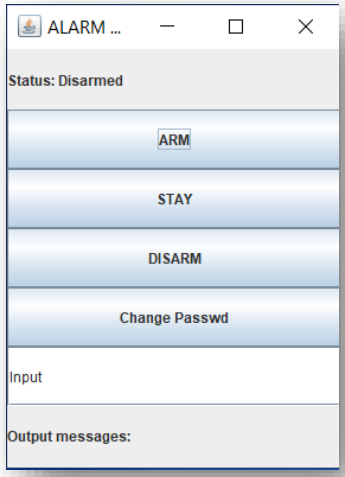
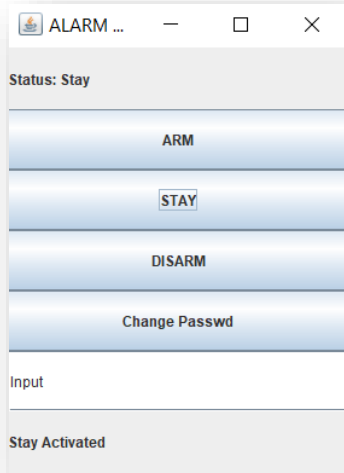
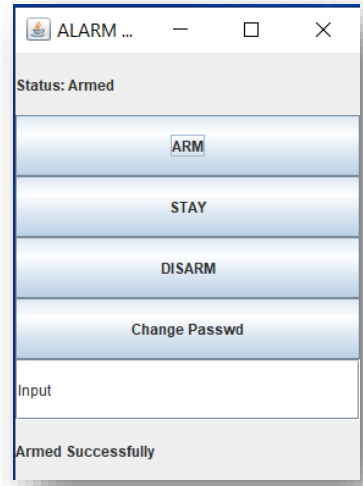
Περισσότερα για τη βιβλιοθήκη jfugue υπάρχουν σε ένα turorial pdf που υπάρχει στο moodle, καθώς στην δνση: http://www.jfugue.org/

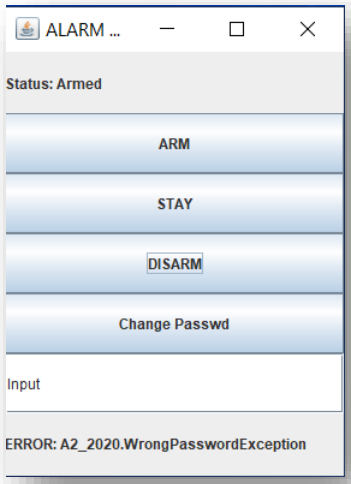
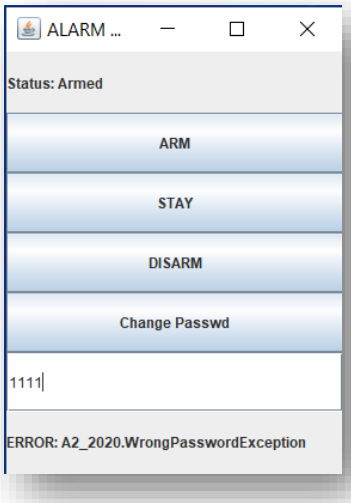
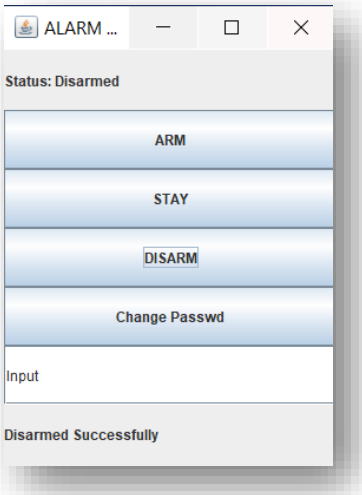
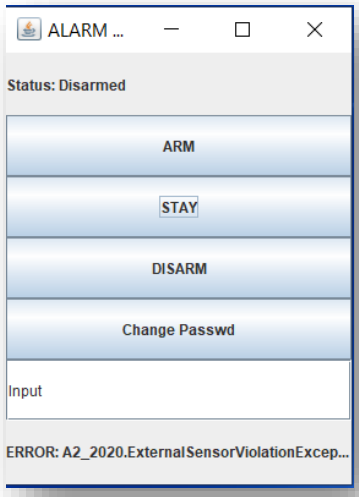
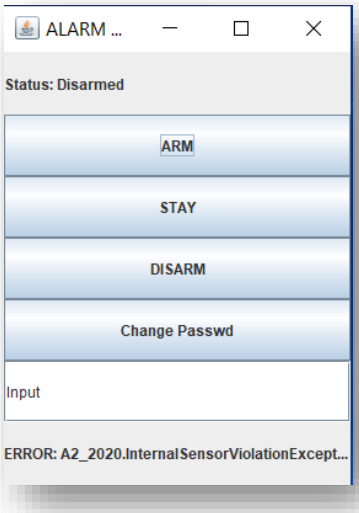
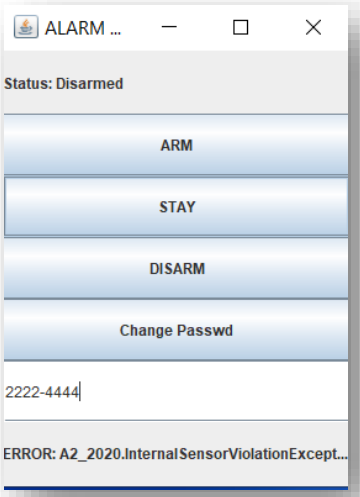
ΠΡΟΑΙΡΕΤΙΚΟ.
Να προστεθεί constructor που να διαβάζει ένα configuration file (όπως θέλετε μπορείτε να το κάνετε) όπου εκεί θα περιγράφονται όλες οι γραμμές και οι αισθητήρες που έχει ο κάθε ένας.
Π.χ.
INTERNALSESORLINE SENSOR S1 false false SENSOR S2 true false SENSOR S3 true true
INTERNALSESORLINE SENSOR S1 false false SENSOR S2 true false
EXTERNALSESORLINE SENSOR S1 false false SENSOR S2 true false SENSOR S12 true true



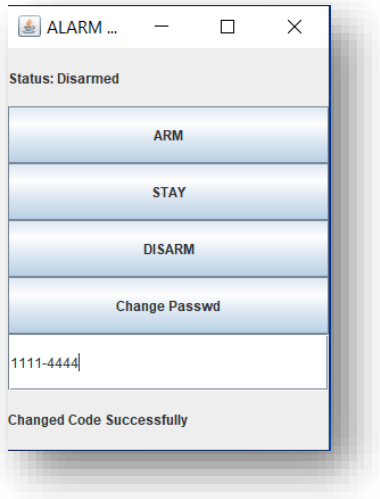
Άσκηση 3. Γραφική Κονσόλα Συστήματος Συναγερμού
Αξία: **20 μονάδες**
Γραφικά στη Java

Καλείστε να φτιάξετε μια απλή γραφική διεπαφή (GUI) που να προσομοιώνει μια κονσόλα συναγερμού και να τη συνδέσετε με τον ελεγκτή συναγερμού που φτιάξατε στην 2η Άσκηση, ώστε με κλικ του ποντικιού ο χρήστης να μπορεί να κάνει Arm, Stay, Disarm και να αλλάξει κωδικό. Η γραφική επαφή μπορεί να είναι της παρακάτω μορφής (δηλαδή να έχει 4 κουμπιά για τις λειτουργίες του ελεγκτή, ένα στοιχείο για είσοδο από το χρήστη, και μια περιοχή για να μηνύματα εξόδου) και πρέπει να λειτουργεί με τον αναμενόμενο τρόπο (τμήμα του κώδικα σας δίνεται), όπως περιγράφεται και στις παρακάτω εικόνες οθόνης:

1/ Αρχική κατάσταση	2/ Μετά από πάτημα στο STAY	3/ Μετά από πάτημα του ARM
		
4/ Μετά από DISARM χωρίς να έχουμε δώσει κωδικό	5/ Γράφουμε το σωστό κωδικό	6/ Μετά από πάτημα στο DISARM (έχοντας γράψει το σωστό κωδικό)

		
7/ Μετά από πάτημα στο STAY αν ένας εξωτερικός αισθητήρας έχει παραβίαση	7/ Μετά από πάτημα του ARM αν ένας εσωτερικός αισθητήρας έχει παραβίαση	8/ Μετά από αίτημα αλλαγής κωδικού (με λάθος αρχικό κωδικό)
		
9/ Μετά από αίτημα αλλαγής κωδικού (έχοντας γράψει το σωστό αρχικό κωδικό)		

Yannis Tzitzikas, Department of Computer Science, University of Crete

Η κλάση που πρέπει να φτιάξετε να έχει το όνομα **AlarmConsole**, και σας δίνεται τμήμα του κώδικά της (στο παράρτημα)

Επίσης σας δίνεται (στο Παράρτημα στο τέλος) κώδικας (**SensorsConsole**) που φτιάχνει γραμμές αισθητήρων και μετά τις παρουσιάζει σε παράθυρο όπου για κάθε μία γραμμή δείχνει τους αισθητήρες της και επιτρέπει στο χρήστη διαδραστικά να αλλάξει την κατάσταση παραβίασης όποιου αισθητήρα επιθυμεί, όπως φαίνεται παρακάτω.



Αυτό θα σας επιτρέψει να δοκιμάσετε εύκολα αν ο συναγερμός δουλεύει καλά σε όλες τις περιπτώσεις: για arm, stay και για παραβιάσεις - αν έχετε υλοποιήσει το προαιρετικό κουδούνισμα πρέπει να λειτουργεί σωστά.
Ο κώδικας επίσης έχει τον σκελετό της συνολικής εφαρμογής (**SensorsGUIapp**), που ξεκινάει τα δύο παράθυρα (AlarmConsole και SensorsConsole με τους αισθητήρες), και υποδεικνύει τα σημεία που πρέπει να συμπληρώσετε.
Θα χρειαστεί να μελετήσετε τις σημειώσεις για τις γραφικές διεπαφές (κάτι που θα σας φανεί χρήσιμο και για την εκπόνηση του project).



Άσκηση 4 Νήματα (στην Κονσόλα Συναγερμού)

Αξία 5 μονάδες

Νήματα (threads)

Στην παρούσα άσκηση καλείστε να εμπλουτίστε την άσκηση με το συναγερμό ώστε η λειτουργικότητα που προσφέρει να είναι όπως στα πραγματικά συστήματα συναγερμού. Συγκεκριμένα, εμπλουτίστε τον κώδικά σας ώστε με Arm ή Stay ο ελεγκτής να ελέγχει ανά 1 δευτερόλεπτο το κατά πόσο δεν υπάρχει κάποια παραβίαση. Αν εντοπιστεί παραβίαση, τότε πρέπει να αρχίζει το κουδούνισμα του συναγερμού. Επίσης το Disarm πρέπει να σταματάει αυτόν τον περιοδικό έλεγχο και να σταματάει το κουδούνισμα του συναγερμού.

Εμπλουτίστε τα tests με ένα test σενάριο όπου ο συναγερμός να ενεργοποιείται, μετά από 3 δευτερόλεπτα ανοίγει η πόρτα, και μετά από 4 δευτερόλεπτα να γίνεται disarm. Αντίστοιχα φτιάξτε και ένα σενάριο για το stay.

Hint: Θα χρειαστεί να χρησιμοποιήσετε **νήματα (threads)** για να το επιτύχετε. Πιθανότατα δεν θα τα έχουμε πει στο μάθημα έως τότε, όμως μπορείτε μόνοι σας να διαβάσετε την αντίστοιχη ενότητα του βιβλίου.



Άσκηση 5. ***Star Wars: Ένας απλός Πυρήνας για Παιχνίδια Ρόλων (core for Role Playing Games)***

Αξία: **35 μονάδες**

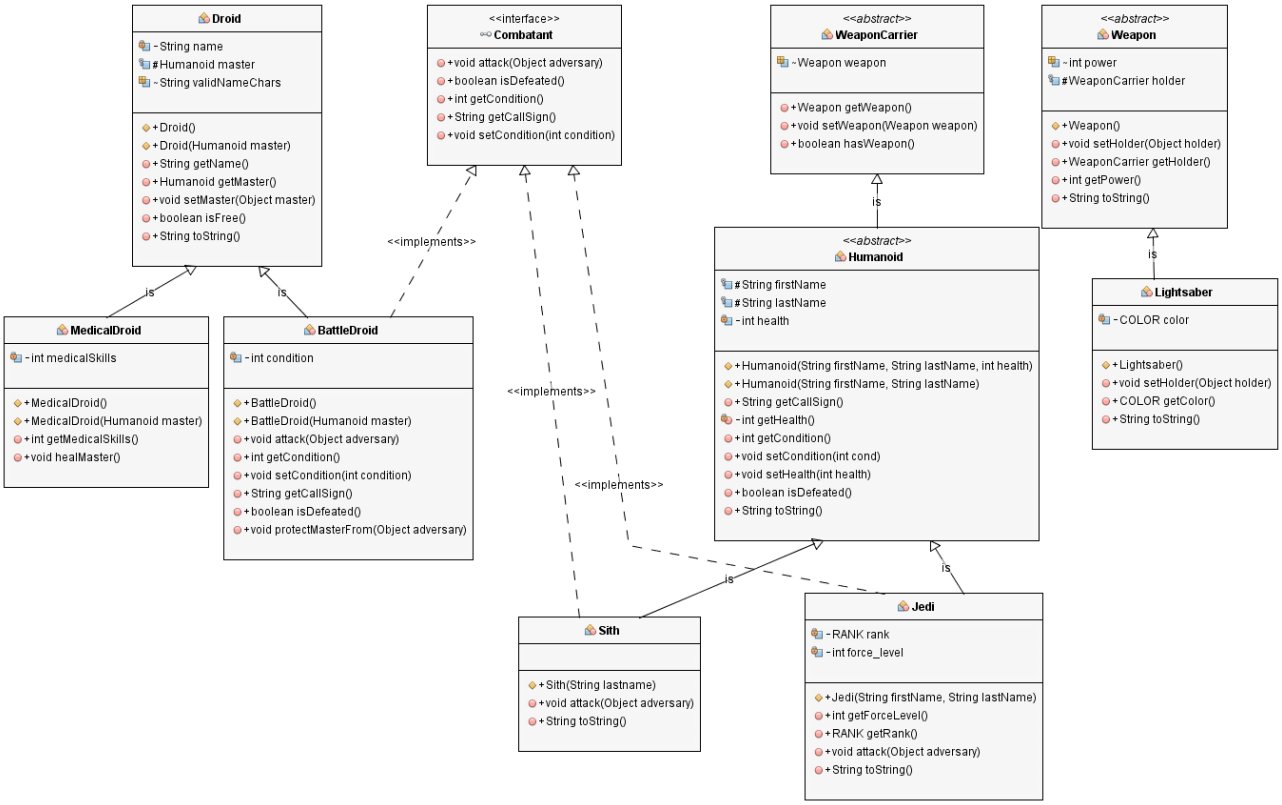
Προδιαγραφή [και Υλοποίηση] Κλάσεων, Εξαιρέσεις, Διεπαφές

Bonus για την καλύτερη εργασία: ένα φωτόσπαθο και ένα εισητήριο για την αντίστοιχη κινηματογραφική ταινία που αναμένεται για τον Δεκέμβριο.

Σε αυτή την άσκηση καλείστε να μοντελοποιήσετε ένα τμήμα του περιβάλλοντος ενός παιχνιδιού ρόλων. Σε αυτό το παιχνίδι εμφανίζονται διάφορες οντότητες από τη σειρά Star Wars, όπως **Jedi**, **Sith** και **Droids**.

Jedi	Sith	Droid
		

Οι **Jedi** και οι **Sith** είναι ανθρωποειδή (**Humanoid**) και έχουν κάποια κοινά χαρακτηριστικά, όπως όνομα, επώνυμο και υγεία. Επιπλέον, οι Jedi έχουν τη δική τους ιεραρχία (*YOUNGLING, PADAWAN, KNIGHT, MASTER, GRAND_MASTER*), η οποία καθορίζεται από το βαθμό στον οποίο έχουν "κατακτήσει" μια υπερφυσική δύναμη (the Force). Τα ρομπότ (**Droid**), τα οποία χωρίζονται σε ιατρικά (**MedicalDroid**) και πολεμικά (**BattleDroid**), έχουν όλα έναν αφέντη (ανθρωποειδές). Τα ιατρικά ρομπότ μπορούν να θεραπεύσουν τον αφέντη τους, ενώ τα πολεμικά ρομπότ μπορούν να προστατεύσουν τον αφέντη τους σε κάποια μάχη. Σε μία μάχη μπορούν να συμμετέχουν μόνο οντότητες που υλοποιούν τη διεπαφή **Combatant** (δηλαδή Humanoids και BattleDroids). Τα ανθρωποειδή εξειδικεύουν την κλάση **WeaponCarrier** και μπορούν να φέρουν όπλο (**Weapon**), το οποίο τα καθιστά πιο ισχυρά στη μάχη, καθώς τους δίνει επιπλέον δύναμη. Ένα είδος όπλου είναι και το φωτόσπαθο (**Lightsaber**), το οποίο ανάλογα με τον ιδιοκτήτη του έχει και διαφορετικό χρώμα (μπλε για Jedi, κόκκινο για Sith). Παρακάτω σας δίνονται ένα ενδεικτικό UML διάγραμμα των ζητούμενων κλάσεων, καθώς και η αναλυτική περιγραφή τους:



Ένα **Humanoid** – abstract class – χαρακτηρίζεται από τα γνωρίσματα:

- firstName – Δεν μπορεί να είναι κενό
- lastName – Δεν μπορεί να είναι κενό
- health – Παίρνει τιμές [0-10]

Το συμβόλαιο του ADT ενός **Humanoid** είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
Humanoid (String firstName, String lastName, int health)	Δημιουργεί ένα στιγμιότυπο ενός Humanoid με ονοματεπώνυμο και υγεία
Humanoid (String firstName, String lastName)	Καλεί τον παραπάνω κονστράκτορα με default υγεία 10
void setHealth(int health), int getHealth()	Θέτει (και επιστρέφει αντίστοιχα) την τιμή του health (προσοχή στους περιορισμούς)
void setCondition(int cond), int getCondition()	Θέτει (και επιστρέφει αντίστοιχα) την κατάσταση του ανθρωποειδούς, όπου κατάσταση ανθρωποειδούς είναι η υγεία του
boolean isDefeated()	Επιστρέφει false αν το ανθρωποειδές είναι ζωντανό, αλλιώς true
String getCallSign()	Επιστρέφει το ονοματεπώνυμο του ανθρωποειδούς
abstract String toString()	Επιστρέφει όλες τις πληροφορίες για ένα ανθρωποειδές

Ένας **Combatant**– interface –πρέπει να υλοποιεί τις παρακάτω μεθόδους:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
void attack(Object adversary)	Επιτίθεται σε έναν αντίπαλο, ο οποίος θα πρέπει και αυτός να είναι Combatant. Φυσικά, μια οντότητα δε μπορεί να επιτεθεί στον εαυτό της. Ο Combatant που κάνει την επίθεση θα πρέπει να έχει κατάσταση > 0. Στο τέλος της επίθεσης θα πρέπει να τυπώνεται ότι η τρέχουσα οντότητα (με το όνομά της) επιτέθηκε στον adversary (με το όνομά του), καθώς και η νέα κατάσταση του adversary. Μετά την επίθεση, η κατάσταση του adversary θα πρέπει να είναι μικρότερη από την προηγούμενή του κατάσταση.
boolean isDefeated()	Επιστρέφει το αν η οντότητα έχει ηττηθεί (δηλαδή η κατάστασή της είναι 0)
int getCondition(), void setCondition(int condition)	Επιστρέφει (και αναθέτει αντίστοιχα) την κατάσταση της οντότητας, η οποία ορίζεται στο [0,10]
String getCallSign()	Επιστρέφει το όνομα το οποίο χρησιμοποιεί η οντότητα στη μάχη

Ένας **Jedi** χαρακτηρίζεται (επιπλέον) από τα γνωρίσματα:

- rank – παίρνει τις τιμές {*YOUNGLING, PADAWAN, KNIGHT, MASTER, GRAND_MASTER*}
- force_level - Παίρνει τιμές [1-100]

Το συμβόλαιο του ADTενός **Jedi** είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
Jedi (String firstName, String lastName)	Δημιουργεί ένα νέο Jedi, με το δοθέν ονοματεπώνυμο και υγεία 10, αναθέτει μια τυχαία τιμή στο force_level και αναθέτει την τιμή του rank, σύμφωνα με την εξής αντιστοίχιση με το force_level: YOUNGLING: [1-10), PADAWAN: [10-30), KNIGHT: [30-70), MASTER: [70-90), GRAND_MASTER: [90-100]
int getForceLevel(), RANK getRank	Επιστρέφουν τις τιμές του force_levelκαι rank, αντίστοιχα
void attack(Object adversary)	Επιτίθεται σε έναν αντίπαλο (ο οποίος δεν επιτρέπεται να είναι Jedi), μειώνοντας την κατάσταση του αντιπάλου. Αν ο Jedi διαθέτει όπλο, τότε η μείωση των πόντων του αντιπάλου είναι όση η δύναμη του όπλου συν όση η ιεραρχία του Jedi (0 για YOUNGLING, 1 για PADAWAN, 2 για KNIGHT, …). Αν ο Jedi δεν έχει όπλο, τότε η μείωση των πόντων του αντιπάλου είναι όση η ιεραρχία του Jedi+1.
String toString()	Επιστρέφει το ονοματεπώνυμο, τη θέση στην ιεραρχία και το όπλο του Jedi

Ένας **Sith** δεν έχει επιπλέον γνωρίσματα.
Το συμβόλαιο του ADTενός **Sith** είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
Sith (String lastName)	Δημιουργεί ένα νέο Sith, με μικρό όνομα «Darth», το δοθέν επώνυμο και υγεία 10
void attack(Object adversary)	Επιτίθεται σε έναν αντίπαλο μειώνοντας την κατάσταση του αντιπάλου. Αν ο Sith διαθέτει όπλο, τότε η μείωση των πόντων του αντιπάλου είναι όση η δύναμη του όπλου. Αν ο Sith δεν έχει όπλο, τότε η μείωση των πόντων του αντιπάλου είναι 1.
String toString()	Επιστρέφει το ονοματεπώνυμο και το όπλο του Sith

Ένα **Weapon** – abstract class – έχει τα γνωρίσματα:

- power - Η δύναμη του όπλου ορίζεται στο [1,5]
- WeaponCarrier holder – Ο κάτοχος του όπλου (η κλάση περιγράφεται παρακάτω)

Το συμβόλαιο του ADTενός **Weapon** είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
Weapon()	Δημιουργεί ένα νέο όπλο με τυχαία δύναμη, χωρίς κάτοχο
intgetPower()	Επιστρέφει τη δύναμη του όπλου
void setHolder(Object holder), WeaponCarrier getHolder()	Αναθέτει τον κάτοχο του όπλου, κάνοντας τους κατάλληλους ελέγχους και, αντίστοιχα, επιστρέφει τον κάτοχο του όπλου
abstract String toString()	Επιστρέφει τις πληροφορίες για ένα όπλο

Ένα **Lightsaber** έχει το γνώρισμα:

- color – μπορεί να πάρει τις τιμές {RED, BLUE}

Το συμβόλαιο ενός **Lightsaber** είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
Lightsaber()	Δημιουργεί ένα νέο φωτόσπαθο με τυχαία δύναμη, χωρίς κάτοχο και χωρίς χρώμα
int getColor()	Επιστρέφει το χρώμα του φωτόσπαθου
void setHolder(Object holder)	Αναθέτει τον κάτοχο του όπλου, κάνοντας τους κατάλληλους ελέγχους και αναθέτοντας το κατάλληλο χρώμα στο φωτόσπαθο
String toString()	Επιστρέφει το χρώμα και τη δύναμη του φωτόσπαθου

Ένας **WeaponCarrier** – abstract class – έχει το γνώρισμα:

- Weapon weapon – ένα όπλο

Το συμβόλαιο του ADT ενός **WeaponCarrier** είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
Weapon getWeapon()	Επιστρέφει το όπλο του WeaponCarrier
Boolean hasWeapon()	Επιστρέφει true αν ο WeaponCarrier έχει όπλο
void setWeapon(Weapon weapon)	Αναθέτει στον WeaponCarrier ένα όπλο (προσοχή στην ανανέωση του πεδίου holder ενός Weapon)

Ένα **Droid**– abstract class – έχει τα γνωρίσματα:

- name – Το όνομα του ρομπότ
- Humanoid master – Ο αφέντης του ρομπότ

Το συμβόλαιο του ADTενός **Droid** είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
Droid()	Δημιουργεί ένα νέο ρομπότ, με όνομα αποτελούμενο από 4 τυχαίους αλφαριθμητικούς χαρακτήρες [A-Z,0-9], χωρίς αφέντη
Droid(Humanoid master)	Καλεί τον παραπάνω κονστράκτορα και επιπλέον αναθέτει έναν αφέντη στο ρομπότ
void setMaster(Object master)	Αναθέτει στο droid έναν αφέντη (ο οποίος δε μπορεί να είναι Droid)
Humanoid getMaster()	Επιστρέφει τον αφέντη του ρομπότ
String getName()	Επιστρέφει το όνομα του ρομπότ
boolean isFree()	Επιστρέφει true αν το ρομπότ δεν έχει αφέντη, ή αν ο αφέντης του έχει ηττηθεί, διαφορετικά false
String toString()	Επιστρέφει το όνομα τουρομπότ και το όνομα του αφέντη του

Ένα **MedicalDroid** έχει τα (επιπλέον) γνωρίσματα:

- medicalSkills – Παίρνει τιμές στο [1,3]

Το συμβόλαιο του ADTενός **MedicalDroid**ε ίναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
MedicalDroid()	Δημιουργεί ένα νέο MedicalDroid με τυχαίο medicalSkills

MedicalDroid(Humanoid master)	Δημιουργεί ένα νέο MedicalDroid με τυχαίο medicalSkills και αναθέτει σε αυτό έναν αφέντη
Humanoid getMedicalSkills()	Επιστρέφει την τιμή του medicalSkills
String getName()	Επιστρέφει το όνομα του ρομπότ
void healMaster()	Βελτιώνει την κατάσταση του αφέντη κατά medicalSkills μονάδες. Προσοχή στους περιορισμούς!

Ένα **BattleDroid** έχει το (επιπλέον)γνώρισμα:

- condition – Παίρνει τιμές στο [0,10]

Το συμβόλαιο του ADTενός **BattleDroid**είναι:

Όνομα μεθόδου	**Περιγραφή μεθόδου**
BattleDroid()	Δημιουργεί ένα νέο BattleDroid με condition 10
BattleDroid(Humanoid master)	Δημιουργεί ένα νέο BattleDroid με condition 10, στο οποίο αναθέτει έναν αφέντη
int getCondition(), void setCondition(int condition)	Επιστρέφει (και αναθέτει αντίστοιχα) την τιμή του condition
String getCallSign()	Επιστρέφει το όνομα του BattleDroid
void attack(Object adversary)	Επιτίθεται σε έναν αντίπαλο μειώνοντας την κατάσταση του αντιπάλου κατά 1.
boolean isDefeated()	Επιτρέφει true αν η τιμή του condition είναι 0, διαφορετικά false
void protectMasterFrom(Object adversary)	Επιτίθεται στον adversary. Προϋποθέτει ότι: - ο αφέντης δεν έχει ηττηθεί -ο adversary μπορεί να επιτεθεί στον αφέντη

Είστε ελεύθεροι να χρησιμοποιήσετε Exceptions, όπου εσείς το κρίνετε.

Σας δίνεται μία main μέθοδος, η οποία θα πρέπει να τρέχει (και να τερματίζει) με τον κώδικά σας, χωρίς λάθη, καθώς και μία ενδεικτική έξοδος αυτής:

```
public static void main(String[] args) {
   Jedi obiwan = new Jedi("Obiwan", "Kenobi");
   obiwan.setWeapon(new Lightsaber());
   System.out.println(obiwan);

   Sith vader = new Sith("Vader");
   vader.setWeapon(new Lightsaber());
   System.out.println(vader);

   MedicalDroiddocDroid = new MedicalDroid();
```

```
    docDroid.setMaster(vader);
    System.out.println(docDroid);

    BattleDroidbattleDroid = new BattleDroid();
    battleDroid.setMaster(obiwan);
    System.out.println(battleDroid);

    //battle between jedi and sith
    try {
        while (!obiwan.isDefeated() && !vader.isDefeated()) {
            if (new Random().nextBoolean()) {               //random turn
                obiwan.attack(vader);
                if (!vader.isDefeated()) {
                    vader.attack(obiwan);
                }
            } else {
                vader.attack(obiwan);
                if (!obiwan.isDefeated()) {
                    obiwan.attack(vader);
                }
            }
            if (!docDroid.isFree()) {
                docDroid.healMaster();
            }
            if (!battleDroid.isFree() && !vader.isDefeated()) {
                battleDroid.protectMasterFrom(vader);
            }
            System.out.println(); //print a new line for a new round
        }
    } catch (Exception e) {
        System.err.println(e);
    } finally {
        System.out.print(obiwan.isDefeated()? vader.getCallSign() : obiwan.getCallSign());
        System.out.println(" won!");
    }
}
```

Ενδεικτικήέξοδος:

```
Jedi:
Obiwan Kenobi
Rank: KNIGHT
Weapon: BLUE lightsaber, of power 2

Sith:
Darth Vader
Weapon: RED lightsaber, of power 1

Droid:
5F2G
Master: Darth Vader

Droid:
N3R7
Master: Obiwan Kenobi

Obiwan Kenobi attacked Darth Vader
Darth Vader's condition:6
Darth Vader attacked Obiwan Kenobi
Obiwan Kenobi's condition:9
```

5F2G healed 3 units of its master Darth Vader
Darth Vader's condition:9
N3R7 attacked Darth Vader
Darth Vader's condition:8
N3R7 was just protecting its master Obiwan Kenobi

Obiwan Kenobi attacked Darth Vader
Darth Vader's condition:4
Darth Vader attacked Obiwan Kenobi
Obiwan Kenobi's condition:8
5F2G healed 3 units of its master Darth Vader
Darth Vader's condition:7
N3R7 attacked Darth Vader
Darth Vader's condition:6
N3R7 was just protecting its master Obiwan Kenobi

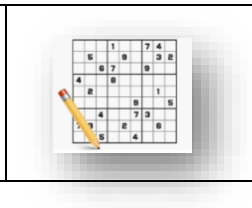
Obiwan Kenobi attacked Darth Vader
Darth Vader's condition:2
Darth Vader attacked Obiwan Kenobi
Obiwan Kenobi's condition:7
5F2G healed 3 units of its master Darth Vader
Darth Vader's condition:5
N3R7 attacked Darth Vader
Darth Vader's condition:4
N3R7 was just protecting its master Obiwan Kenobi

Darth Vader attacked Obiwan Kenobi
Obiwan Kenobi's condition:6
Obiwan Kenobi attacked Darth Vader
Darth Vader's condition:0

Obiwan Kenobi won!

Παραδοτέα (10 αρχεία .java):Δημιουργήστε ένα φάκελο A23 και μέσα 2 υποφακέλους creatures (η οποία θα περιέχει τα java αρχεία: BattleDroid.java, Combatant.java, Droid.java, Humanoid.java, Jedi.java, MedicalDroid.java, Sith.java, WeaponCarrier.java) και weapons (η οποία θα περιέχει τα java αρχεία: Lightsaber.java, Weapon.java).

ΠΡΟΑΙΡΕΤΙΚΗ BONUS Άσκηση

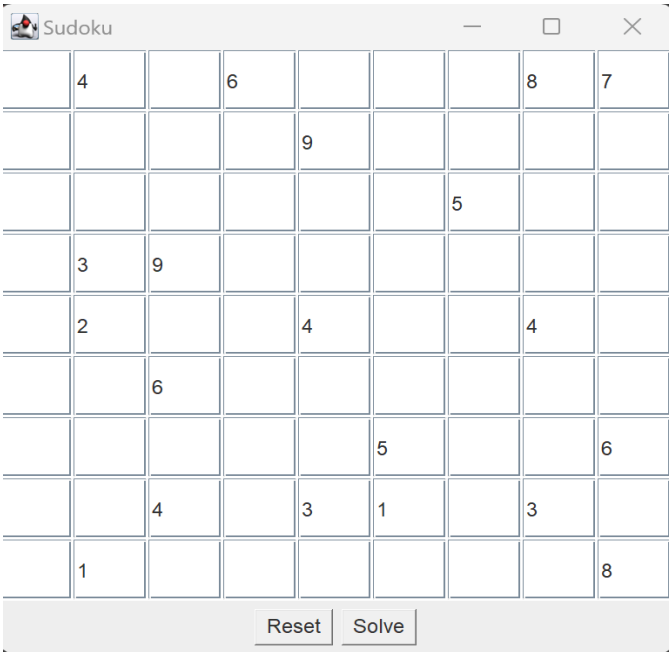



Άσκηση 6. Εισαγωγή στην ανάπτυξη native Android εφαρμογών
Αξία: 20 **μονάδες (bonus)**

(α) MVC [5 μονάδες]
Μελετήστε την αρχιτεκτονική Model-View-Controller (MVC) για να μετατρέψετε τον κώδικα για Sudoku που γράψατε στην A1 σε μια πλήρη εφαρμογή ακολουθώντας την αρχιτεκτονική MVC[1]. Συγκεκριμένα,

[1] Σύμφωνα με αυτό το πρότυπο θα πρέπει να διαχωρίζεται η ανάπτυξη της γραφικής διεπαφής του παιχνιδιού (View) από τον πυρήνα του παιχνιδιού που περιέχει όλη την πληροφορία κατάστασης (Model) και από τον μηχανισμό διαχείρισης και ενημέρωσης των ενεργειών του παιχνιδιού με τη γραφική του απεικόνιση (Controller). Δείτε ένα video για να το καταλάβετε πλήρως:

- Φτιάξτε ένα αρχείο view.java που να δημιουργεί ένα grid με 9χ9 text boxes που θα αντιπροσωπεύει το board του Sudoku. Προσθέστε στο GUI ένα κουμπί solve που θα επιλύει το παζλ δείχνοντας την λύση πάνω στον πίνακα (θα συμπληρώνει τα κενά κελιά όπως στην Α1) και ένα κουμπί reset που θα καθαρίζει τον πίνακα (όλα τα συμπληρωμένα κελιά θα γίνονται κενά).



Ενδεικτική υλοποίηση του view.java με 20 συμπληρωμένα κελιά

- Δημιουργήστε το πακέτο Model της εφαρμογής που θα προσφέρει την μέθοδο solve, όπως την υλοποιήσατε στην Α1. Εμπλουτίστε το Model με τη μέθοδο reset με τη λειτουργία που περιγράφεται παραπάνω.

- Υλοποιήστε την κλάση Controller της εφαρμογής η οποία θα γνωρίζει τις προηγούμενες 2 κλάσεις και
  - Θα διαβάζει το input του χρήστη (από το view)
  - Θα καλεί τη solve (του Model) όποτε κληθεί από τον χρήστη, και θα ενημερώνει το gui (View) με τη λύση
  - Θα καλεί τη reset όποτε κληθεί από τον χρήστη.

Hint

Μπορείτε να συμβουλευτείτε το παράδειγμα MVC που βρίσκεται στο github καθώς και τα σχετικά παραδείγματα των φροντιστηρίων (π.χ. week 9 – Τρίλιζα).

(β) Android Application: Αρχική Σελίδα [5 μονάδες bonus]

Για όσους ενδιαφέρονται, θα μπορούσαν από τώρα να διαβάσουν τις σχετικές διαφάνειες για Android development (και το σχετικό κεφάλαιο του βιβλίου), να ακολουθήσουν τις οδηγίες που αναγράφονται εκεί και να πάρουν μια πρώτη γεύση από την ανάπτυξη εφαρμογών Android σε Java. Σκοπός της

http://www.newthinktank.com/2013/02/mvc-java-tutorial/

άσκησης είναι να δημιουργήσετε μια απλή Android εφαρμογή για Sudoku, αξιοποιώντας τον κώδικα που έχετε ήδη γράψει στην Α1.
Υλοποιήστε την αρχική σελίδα (main menu) της εφαρμογής σας. Η σελίδα θα πρέπει να περιέχει τα εξής:

- Τον τίτλο του παιχνιδιού. Μπορείτε να δώσετε κάποιο δικό σας όνομα (π.χ. MySudoku).
- Το logo του παιχνιδιού. Μπορείτε να κατεβάσετε μια δωρεάν εικόνα από το internet και να την φορτώσετε στην εφαρμογή σας.
- Κουμπί solve που θα ξεκινάει την εφαρμογή με τη λειτουργία που περιγράφεται στο ερώτημα γ (Επίλυση παζλ).
- Κουμπί exit που θα τερματίζει την εφαρμογή.

γ) Android Application: Επίλυση παζλ [10 μονάδες bonus]
Αξιοποιείστε τον κώδικα που έχετε ήδη γράψει στην Α1 για Sudoku, προκειμένου να υλοποιήσετε την λειτουργία που θα δημιουργεί ένα πίνακα με ορισμένο αριθμό συμπληρωμένων κελιών και θα επιτρέπει στο χρήστη να συμπληρώσει τα κενά κελιά και να λύσει το παζλ αντίστοιχα πατώντας το κουμπί solve (ή το πλήκτρο Enter), όπως το κλασσικό Sudoku, όπου θα ελέγχεται η ορθότητα της λύσης του χρήστη.

Σημείωση: Είστε ελεύθεροι να υλοποιήσετε τα ζητούμενα με όποιον τρόπο επιθυμείτε και παροτρύνεστε να πειραματιστείτε με τις διάφορες πιθανές λύσεις (Layouts, Activities, etc.).

Προαιρετικά
Προαιρετικά μπορείτε να εμπλουτίστε την εφαρμογή με ό,τι άλλο επιθυμείτε.

Μπορείτε, για παράδειγμα, να δώσετε λίγο χρώμα και style στην εφαρμογή ώστε να γίνει πιο φιλική προς το χρήστη (user friendly) και ελκυστική. Τέτοιες πινελιές είναι για παράδειγμα τα χρώματα, τα fonts, τα styles κ.α.

Άλλο παράδειγμα: Η εφαρμογή θα μπορούσε να περιέχει μενού για settings ή game modifications όπου ο χρήστης θα μπορεί να επιλέξει συγκεκριμένα χαρακτηριστικά και περιορισμούς για την εμπειρία του με αυτήν (π.χ. να αλλάξει τα χρώματα της εφαρμογής).

Καλή εργασία!

Παράρτημα για την Άσκηση 4

Κώδικας:

```
import java.awt.Color;
import java.awt.GridLayout;
import java.awt.event.ActionListener;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

/**
 * Takes as input an array of sensorlines and for each one of them
```

```
 * it shows each sensor as a button that toggles their violation status
 * @author tzitzik
 *
 */
class SensorsConsole extends JFrame {
        SensorLine[] sensorLines;

        void fill() {
                for (int i=0;i<sensorLines.length;i++) { // for each sensor line
                        final SensorLine sl = sensorLines[i];
                        JPanel sensorlinePanel = new JPanel(new GridLayout(0,4,5,5)); // rows, columns, int hgap,
int vgap)
                        sensorlinePanel.setBorder(BorderFactory.createTitledBorder(
                                BorderFactory.createEtchedBorder(), sl.getName()));
                        //add(new JLabel(sl.getName()));
                        for (final Sensor s: sl.getSensors()) {  // gets the  array of active sensors
                                final JButton t = new JButton(s.getId()); // creation of button
                                t.setToolTipText(s.toString());
                                t.setOpaque(true);
                                t.setBackground(s.isViolation()? Color.RED: Color.GREEN);
                                t.addActionListener(  // addition of the behavior of the button
                                        new ActionListener() {
                                                public void actionPerformed
                                                        (java.awt.event.ActionEvent e) {
                                                                s.setOn(true); // optional: sets the sensor on
                                                                s.setViolation(!s.isViolation()); // toggles the
violation status

                                                                t.setLabel(s.getId()); // updates the button's

label

                                                                t.setBackground(s.isViolation()? Color.RED:

Color.GREEN);

                                                                t.setToolTipText(s.toString());
                                                }
                                        }
                                );
                                sensorlinePanel.add(t); // adds the button to the panel
                                add(sensorlinePanel); // adds the panel  to the jframe
                        }
                }
                setVisible(true); // makes the jframe visible
        }


        SensorsConsole(String title, SensorLine[] sensorLines) {
                this.sensorLines = sensorLines;
                this.setTitle(title);

                setBounds(200,100,800,600); //x,  y,  width,  height)
                setLayout(new GridLayout(0,1)); // rows, columns
                setVisible(true);
                fill();

        }
}

/**
 * Graphical User Interface for the Alarm Controller
 * @author tzitzik
 *
 */
class AlarmConsole extends JFrame {
        HomeSecurity homeSecurityController ;
        JLabel labelStatus ;
        JLabel labelLog ;
        JButton buttonArm ;
        JButton buttonStay;
        JButton buttonDisarm ;
        JButton buttonChangeCode ;
        JTextField inputField;


        AlarmConsole(HomeSecurity homeSecurityController ) {

ΣΥΜΠΛΗΡΩΣΤΕ ΩΣΤΕ ΝΑ ΔΗΜΙΟΥΡΓΟΥΝΤΑΙ ΤΑ ΚΟΥΜΠΙΑ  ΠΟΥ ΠΡΕΠΕΙ, ΚΑΙ ΤΟ ΠΑΤΗΜΑ ΤΟΥΣ ΝΑ ΚΑΝΕΙ ΤΙΣ
ΚΑΤΑΛΛΗΛΕΣ ΚΛΗΣΕΙΣ στον homeScurityController (που έχετε υλοποιήσει από τη 2η άσκηση).


        }
}


//====================

/**
 * The entire application comprising sensor simulator and graphical alarm console
 * @author tzitzik
 *
 */
class SensorsGUIapp {


        public static void main(String[] args) {
                System.out.println("SENSORS GUI");
```

```
                // A. CREATION OF SENSORS
                int K = 3; // number of sensor lines to be created
                int M = 4; // number of sensors per line
                SensorLine[] internallines = new SensorLine[K]; // internal sensor lines
                SensorLine[] externallines = new SensorLine[K]; // external sensor lines
                // creation of internal and sensor lines
```

ΣΥΜΠΛΗΡΩΣΤΕ ΩΣΤΕ ΝΑ ΔΗΜΙΟΥΡΓΟΥΝΤΑΙ ΟΙ ΑΙΣΘΗΤΗΡΕΣ ΚΑΙ ΝΑ ΔΕΙΧΝΟΝΤΑΙ ΑΠΟ ΤΟΥΣ ΠΑΡΑΠΑΝΩ ΔΥΟ ΠΙΝΑΚΕΣ

```
                // B. PASSES THE ARRAY OF SENSORS TO TWO SensorConsoles

                SensorsConsole internalSensorsConsole = new SensorsConsole("Internal Sensor Lines", internallines);
                SensorsConsole externalSensorsConsole = new SensorsConsole("External Sensor Lines", externallines);

                // C. CREATION OF HomeSecurityController (from Askisi 2)
                HomeSecurity homeSecurityController = new HomeSecurity(internallines,externallines);

                // D. PASSES HomeSecurityController TO AlarmConsole
                AlarmConsole  alarmConsole = new AlarmConsole(homeSecurityController);

        }

}
```

Τέλος Α2